AI speed. Human accountability.

CREDSHIELDS · SMART CONTRACT AUDIT

Smart Contract Audit

CarbonFi Labs

Confidential — Prepared for the CarbonFi Engineering Team

This report presents the findings of a smart contract security audit conducted against CarbonFi Smart Contracts. Findings were identified through manual review of the contract source by the CredShields' security team.

2	2	8	15	11	38
CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL FINDINGS

CredShields Technologies Pte. Ltd. · Singapore
credshields.com · info@credshields.com

CONFIDENTIAL

/ CONTENTS

Table of Contents

01 Executive Summary

02 Vulnerability Summary

03 Detailed Findings

C001	Staked Principal Is Permanently Locked No Unstake / Withdraw-Principal Function	CRITICAL
C002	Reward-Pool Drain claimReward Over-Accrues on Every Pre-Unlock Claim	CRITICAL
H001	Principal Permanently Locked via timeHeld Underflow (Non-Auto Stake)	HIGH
H002	Founder's 90% Retirement Fees Permanently Locked in withdrawPendingFees	HIGH
M001	Listing Fee Evasion via Unvalidated Seller-Declared totalCo2eTon	MEDIUM
M002	Signature Orders Cannot Be Cancelled and Have Unbounded Expiry	MEDIUM
M003	Reward-Pool Double-Counting and Incorrect Require Cause Claim DoS / Stuck Rewards	MEDIUM
M004	Compounded Rewards Permanently Forfeited When Auto-Staking Is Toggled Off Before Claim	MEDIUM
M005	Escrowed NFT and Locked CAFI Trapped When a Credit Expires Mid-Listing	MEDIUM
M006	Listing Fee Divided by 1e18 Approximately 1e18× Too Small	MEDIUM
M007	Partial Fills Unfillable Maker Balance Checked Against Full order.amount	MEDIUM
M008	Compounded Rewards Permanently Forfeited When Unstaking a Matured Stake With Zero Fresh Reward	MEDIUM
L001	Missing Handling of Fees on Transfer in ERC20 Token Transfers	LOW
L002	Single-Step Ownership Transfer (Ownable instead of Ownable2Step)	LOW
L003	Carbon Credit Expiry (end) Stored but Never Enforced	LOW
L004	Non-Unique Retirement Certificate ID Allows Record Overwrite	LOW
L005	Floating and Outdated Pragma	LOW
L006	CEI Violation in buyItem and cancelListing (NFT Transfer Before State Update)	LOW
L007	Missing events in important functions	LOW
L008	Buy Fee on buyItem Is Borne by the Seller and Stacks on the Seller's Listing Fee	LOW
L009	Unstake Couples Principal Recovery to Reward-Pool Solvency	LOW

L010	Pause Not Enforced on NFT Transfers	LOW
L011	Wallet Rotation Strands Already-Accrued Pending Fees	LOW
L012	MAX_STAKES_PER_USER Counts Historical Stakes Permanent Staking Lockout	LOW
L013	Compound Fee Over-Applied and Trivially Evadable	LOW
L014	emergencyWithdrawExcessRewardToken Can Sweep User-Owed and Staked Tokens	LOW
L015	Compound Fee Bypassable via toggleAutoStake Settlement	LOW
I001	Dead Code	INFO
I002	removeNFTContract Leaves Stale Entries in nftContractList	INFO
I003	Incomplete / Dead Code in CarbonDEX	INFO
I004	buyItem Missing amount > 0 Check (No-op Sales / Event Spam)	INFO
I005	CarbonFiRetirement setRetireFee Lacks Input Validation / Sanity Bounds	INFO
I006	AutoStaking APY Rate Not Snapshotted setAPY Retroactively Re-Prices Existing Stakes	INFO
I007	Marketplace ItemSold Emits pricePerItem = 0 on Full Buyout (Event Field Read After delete)	INFO
I008	AutoStaking Minor Issues (onlyEOA Anti-Pattern, Dead Code, Reward-Token Footgun)	INFO
I009	getActiveStakes Returns Inactive Stakes (Misnamed)	INFO
I010	renounceOwnership Not Disabled on Staking / Marketplace / DEX	INFO
I011	Additional Minor & Informational Observations	INFO

– Lead Reviewers & Signatories

/ 01: THE PICTURE AT A GLANCE

Executive Summary

A security audit was conducted on the CarbonFi Smart Contracts, identifying a total of **38 findings**. These were categorized by severity as: 2 Critical, 2 High, 8 Medium, 15 Low, and 11 Informational.

The two **Critical** findings highlight significant risks. "[C001] Staked Principal Is Permanently Locked No Unstake / Withdraw-Principal Function" indicates that staked principal funds are currently irretrievable. The second critical issue, "[C002] Reward-Pool Drain claimReward Over-Accrues on Every Pre-Unlock Claim," poses a severe risk of the reward pool being drained due to over-accrual.

Two **High** severity issues also require prompt attention. "[H001] Principal Permanently Locked via timeHeld Underflow (Non-Auto Stake)" describes another scenario leading to principal being permanently locked. Additionally, "[H002] Founder's 90% Retirement Fees Permanently Locked in withdrawPendingFees" prevents the withdrawal of a substantial portion of founders' accumulated fees.

Following remediation efforts, a retest confirmed that **36 of the 38 findings have been fixed**, with the remaining 2 findings acknowledged by CarbonFi Labs.

Audited scope

https://github.com/aresa1801/cafi-contract/tree/e50ab5cfda1fb59875372e72a9c718e3f443bd01 (CarbonFi, CarbonFi_Retire, Staking, CarbonFi_Marketplace, Mint_NFT, and CarbonDex)

Retested scope

https://github.com/aresa1801/cafi-contract/tree/a1d7bc5ef64e03cbc6812d5c15d349a492af89cd

Assessment overview

This engagement involved a comprehensive security audit of the CarbonFi Smart Contracts, focusing on identifying potential vulnerabilities, security weaknesses, and logical flaws within the codebase. Our review employed a combination of manual code inspection, static analysis tools, and an understanding of common attack vectors relevant to smart contract environments. The assessment aimed to ensure the robustness, integrity, and adherence to intended business logic of the CarbonFi ecosystem, covering aspects such as tokenomics, staking mechanisms, marketplace functionality, and overall system security.

During the audit, CredShields identified a total of **38 findings**, categorized across various severity levels: 2 Critical, 2 High, 8 Medium, 15 Low, and 11 Informational. We are pleased to report that following our initial assessment and subsequent remediation efforts by the CarbonFi Labs team, **36 of these findings have been fixed**, with the remaining 2 acknowledged. This demonstrates a strong commitment to security and a proactive approach to addressing identified weaknesses.

The most severe issues identified were **two Critical findings**. **[C001] Staked Principal Is Permanently Locked No Unstake / Withdraw-Principal Function** highlighted a fundamental design flaw where users were unable to retrieve their principal stake, leading to permanent loss of funds. The second critical finding, **[C002] Reward-Pool Drain claimReward Over-Accrues on Every Pre-Unlock Claim**, identified a critical business logic error that allowed users to repeatedly claim excessive rewards before their stake matured, potentially draining the reward pool and undermining the system's economic stability.

Further impacting fund safety and protocol integrity were **two High severity findings**. **[H001] Principal Permanently Locked via timeHeld Underflow (Non-Auto Stake)** exposed another mechanism for permanent principal loss, this time due to an integer underflow vulnerability specific to non-auto-staked positions. Additionally, **[H002] Founder's 90% Retirement Fees Permanently Locked in withdrawPendingFees** indicated a significant issue where the founder's accrued fees could become permanently inaccessible, impacting expected revenue streams.

The audit also uncovered **eight Medium severity findings**, predominantly related to business logic errors and a lack of robust input validation. Notable issues included **[M001] Listing Fee Evasion via Unvalidated Seller-Declared totalCo2eTon**, which could allow sellers to avoid fees, and **[M003] Reward-Pool Double-Counting and Incorrect Require Cause Claim DoS / Stuck Rewards**, which could lead to denial-of-service for reward claims or trapped funds. Other medium findings such as **[M004] Compounded Rewards Permanently Forfeited When Auto-Staking Is Toggled Off Before Claim** and **[M005] Escrowed NFT and Locked CAFI Trapped When a Credit Expires Mid-Listing** pointed to scenarios where user funds or assets could become permanently or temporarily inaccessible due to complex interaction logic.

Finally, **fifteen Low severity findings** and **eleven Informational findings** were identified. These ranged from minor business logic inconsistencies like **[L003] Carbon Credit Expiry (end) Stored but Never Enforced** and **[L009] Unstake Couples Principal Recovery to Reward-Pool Solvency**, to best practice recommendations, code quality improvements, and potential gas optimizations. While these findings carry a lower immediate risk, addressing them contributes to the overall robustness, maintainability, and user experience of the CarbonFi ecosystem. The prompt remediation of the vast majority of findings demonstrates CarbonFi Labs' dedication to delivering a secure and reliable platform.

ENGAGEMENT DETAILS

CLIENT	CarbonFi Labs
ASSESSMENT TYPE	Smart Contract Security Audit
AUDIT	CarbonFi Smart Contracts
REPORT DATE	01 Jul 2026
START DATE	10 Jun 2026
END DATE	15 Jun 2026
RETEST DATE	29 Jun 2026
TOTAL FINDINGS	38
REVIEWER	Shashank (shashank@credshields.com) · CredShields

/ FINDING SEVERITY DISTRIBUTION

2	2	8	15	11
CRITICAL	HIGH	MEDIUM	LOW	INFO



/ 02: AT A GLANCE

Vulnerability Summary

Every finding from this assessment, ordered by severity. Follow an ID for the full write-up.

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
C001	Staked Principal Is Permanently Locked No Unstake / Withdraw-Principal Function	CRITICAL	Business Logic (SC02-LogicErrors)	✓ FIXED
C002	Reward-Pool Drain claimReward Over-Accrues on Every Pre-Unlock Claim	CRITICAL	Business Logic (SC02-LogicErrors)	✓ FIXED
H001	Principal Permanently Locked via timeHeld Underflow (Non-Auto Stake)	HIGH	Integer Overflow and Underflow (SC09-Integer Overflow and Underflow)	✓ FIXED
H002	Founder's 90% Retirement Fees Permanently Locked in withdrawPendingFees	HIGH	Business Logic (SC02-LogicErrors)	✓ FIXED
M001	Listing Fee Evasion via Unvalidated Seller-Declared totalCo2eTon	MEDIUM	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
M002	Signature Orders Cannot Be Cancelled and Have Unbounded Expiry	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M003	Reward-Pool Double-Counting and Incorrect Require Cause Claim DoS / Stuck Rewards	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M004	Compounded Rewards Permanently Forfeited When Auto-Staking Is Toggled Off Before Claim	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M005	Escrowed NFT and Locked CAFI Trapped When a Credit Expires Mid-Listing	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M006	Listing Fee Divided by 1e18 Approximately 1e18× Too Small	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M007	Partial Fills Unfillable Maker Balance Checked Against Full order.amount	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M008	Compounded Rewards Permanently Forfeited When Unstaking a Matured Stake With Zero Fresh Reward	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
L001	Missing Handling of Fees on Transfer in ERC20 Token Transfers	LOW	Missing functionality (SCWE-006)	✓ FIXED
L002	Single-Step Ownership Transfer (Ownable instead of Ownable2Step)	LOW	Best Practice	✓ FIXED
L003	Carbon Credit Expiry (end) Stored but Never Enforced	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
L004	Non-Unique Retirement Certificate ID Allows Record Overwrite	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L005	Floating and Outdated Pragma	LOW	Floating Pragma (SCWE-060)	ACCEPTED RISK
L006	CEI Violation in buyItem and cancelListing (NFT Transfer Before State Update)	LOW	Checks-Effects-Interactions	✓ FIXED
L007	Missing events in important functions	LOW	—	✓ FIXED
L008	Buy Fee on buyItem Is Borne by the Seller and Stacks on the Seller's Listing Fee	LOW	Business Logic (SC02-LogicErrors)	ACCEPTED RISK
L009	Unstake Couples Principal Recovery to Reward-Pool Solvency	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L010	Pause Not Enforced on NFT Transfers	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L011	Wallet Rotation Strands Already-Accrued Pending Fees	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L012	MAX_STAKES_PER_USER Counts Historical Stakes Permanent Staking Lockout	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L013	Compound Fee Over-Applied and Trivially Evadable	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L014	emergencyWithdrawExcessRewardToken Can Sweep User-Owed and Staked Tokens	LOW	Access Control (SCWE-016)	✓ FIXED
L015	Compound Fee Bypassable via toggleAutoStake Settlement	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
I001	Dead Code	INFO	—	✓ FIXED
I002	removeNFTContract Leaves Stale Entries in nftContractList	INFO	Code Quality	✓ FIXED
I003	Incomplete / Dead Code in CarbonDEX	INFO	Code Quality	✓ FIXED
I004	buyItem Missing amount > 0 Check (No-op Sales / Event Spam)	INFO	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
I005	CarbonFiRetirement setRetireFee Lacks Input Validation / Sanity Bounds	INFO	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
I006	AutoStaking APY Rate Not Snapshotted setAPY Retroactively Re-Prices Existing Stakes	INFO	Best Practice	✓ FIXED
I007	Marketplace ItemSold Emits pricePerItem = 0 on Full Buyout (Event Field Read After delete)	INFO	Data Integrity	✓ FIXED

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
I008	AutoStaking Minor Issues (onlyEOA Anti-Pattern, Dead Code, Reward-Token Footgun)	INFO	Code Quality	✓ FIXED
I009	getActiveStakes Returns Inactive Stakes (Misnamed)	INFO	Code Quality	✓ FIXED
I010	renounceOwnership Not Disabled on Staking / Marketplace / DEX	INFO	Access Control (SCWE-016)	✓ FIXED
I011	Additional Minor & Informational Observations	INFO	Code Quality / Best Practice	✓ FIXED

/ 03: DETAILED FINDINGS

38 findings across the assessment.

Findings are ordered by severity.

C001 · BUSINESS LOGIC (SC02-LOGICERRORS)

CRITICAL

FIXED

Staked Principal Is Permanently Locked No Unstake / Withdraw-Principal Function

DESCRIPTION

AutoStaking accepts deposits via stake(), which pulls stakingToken into the contract and records info.amount, but the contract contains no function that returns the staked principal. An enumeration of all functions confirms the only token-out path is withdrawRewards, which transfers pendingRewardWithdrawals[msg.sender] in rewardToken; that mapping is credited only in claimReward, and only with reward values (reward, or compoundedAmount + reward - fee, plus fee) never with info.amount. claimReward marks the stake claimed = true and pays only rewards. No admin/emergency function withdraws stakingToken, and totalStaked is only ever incremented. Therefore, once a user stakes, neither the user nor the owner can ever recover the principal it is permanently frozen. This affects 100% of stakes under normal use and requires no attacker interaction. (The sibling contract Farming.sol includes the missing withdrawStake logic, confirming this is an omission rather than intent.)

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L97-L122

BUSINESS IMPACT

Every user permanently and unconditionally loses their entire staked principal; only the small APY reward is ever recoverable and there is no recovery path for either the user or the owner.

RECOMMENDED NEXT STEP

Add a withdrawStake(stakeIndex) callable after unlockTime that transfers info.amount of stakingToken back to the staker, decrements totalStaked, and closes the stake (guard against double-withdrawal).

RETEST

Fixed. An unstake(stakeIndex) function was added (Staking.so): callable after unlockTime, it returns info.amount of stakingToken, decrements totalStaked, and guards double-withdrawal via isActive.

C002 · BUSINESS LOGIC (SC02-LOGICERRORS)

CRITICAL

Reward-Pool Drain claimReward Over-Accrues on Every Pre-Unlock Claim

✓ FIXED

DESCRIPTION

claimReward requires only that the stake is active it has no unlock-time guard and the reward helper _calculateAccruedRewards measures elapsed time against the fixed info.unlockTime rather than the current block (endTime = info.unlockTime). As a result a staker who claims before unlock is paid the entire remaining lock-period reward, not the small amount actually accrued. Because each claim only sets info.lastRewardClaimTime = block.timestamp (which does not lower the fixed endTime), repeated claims in successive blocks, or even within a single transaction re-pay the overlapping [lastClaim, unlockTime] window again and again, draining the shared rewardPoolBalance while the attacker's principal stays locked. The removal of the onlyEOA restriction (original I008 fix) lets a contract loop the claim in one transaction.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L183-L209
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L259-L274

BUSINESS IMPACT

Any single staker can drain the entire reward-token pool funds that belong to all stakers and recover their principal afterward, with no attacker-side risk. Even a one-shot pre-unlock claim over-pays (full future-period yield for zero elapsed time), so reward accounting is broken even without a deliberate attack.

RECOMMENDED NEXT STEP

Add an unlock guard to claimReward so rewards can never be claimed before unlockTime, consistent with compoundReward and unstake. In _calculateAccruedRewards, bound the accrual end (endTime) so it can never run past the current block, with the two stake types handled differently: for non-auto stakes, cap endTime at the earlier of the current block timestamp and unlockTime so accrual stops at unlock and never pays for not-yet-elapsed (future) time; for auto-staking stakes, keep endTime at the current block timestamp both before and after unlock, so auto-compounders continue to earn for the full elapsed period past unlock and lose no reward. With this bound in place, a claim can never exceed the time actually elapsed since lastRewardClaimTime, which closes the repeatable drain. This change also resolves H001.

RETEST

Fixed. claimReward now requires block.timestamp >= info.unlockTime (Staking.sol), and _calculateAccruedRewards bounds endTime correctly block.timestamp for auto stakes, the earlier of block.timestamp/unlockTime for non-auto with if (endTime <= info.lastRewardClaimTime) return 0; (Staking.sol). Pre-unlock claims and the repeated-overlap drain are both eliminated; auto-compounders still earn the full elapsed window. (commit 5e10d0a)

H001 · INTEGER OVERFLOW AND UNDERFLOW (SC09-INTEGER OVERFLOW AND UNDERFLOW)

HIGH

FIXED

Principal Permanently Locked via timeHeld Underflow (Non-Auto Stake)

DESCRIPTION

For a non-auto stake, endTime is frozen at info.unlockTime (the auto-only extension never applies). A normal sequence a staker calls claimReward after the unlock time to collect rewards sets info.lastRewardClaimTime = block.timestamp, which is now greater than unlockTime. On any subsequent call, _calculateAccruedRewards computes timeHeld = endTime - info.lastRewardClaimTime = unlockTime - lastRewardClaimTime, which underflow-reverts under Solidity 0.8.24 checked arithmetic. Because unstake calls _calculateAccruedRewards unconditionally before it can mark the stake inactive, and unstake is the only function that returns the staked principal, every subsequent unstake and claimReward for that stake reverts, blocking principal withdrawal. The lock is not strictly permanent: calling toggleAutoStake to set autoStaking = true makes endTime float to block.timestamp (so endTime > lastRewardClaimTime), eliminating the underflow and letting unstake succeed. That escape is undocumented and counter-intuitive, however, so a normal user who claims after unlock and then tries to unstake hits an unexpected revert on the only principal-return path and would reasonably conclude their principal is lost.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L259-L274
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L151-L180
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L183-L209

BUSINESS IMPACT

A staker who performs the entirely normal action of claiming rewards after unlock can no longer withdraw their principal through the normal flow every unstake/claimReward reverts. Recovery is possible only via an undocumented, counter-intuitive workaround (toggleAutoStake to autoStaking = true, which floats endTime to block.timestamp and clears the underflow); absent that knowledge the principal appears permanently locked, and there is no admin rescue path. This shares its root cause with C002 (endTime pinned to unlockTime).

RECOMMENDED NEXT STEP

Apply the same accrual bound described in C002: set endTime to the current block timestamp for auto stakes, or to the earlier of the current timestamp and unlockTime for non-auto stakes, and return a zero reward whenever endTime is at or before lastRewardClaimTime instead of performing the subtraction. This prevents the underflow and keeps the principal withdrawable, while preserving auto-compounders' post-unlock rewards. Note that adding only the C002 unlock guard without this accrual bound would convert the over-pay into this principal-lock, so the unlock guard and the accrual bound must be applied together.

RETEST

Fixed. The same _calculateAccruedRewards bound plus if (endTime <= info.lastRewardClaimTime) return 0; (Staking.sol) removes the timeHeld underflow, so unstake/claimReward no longer revert and principal is always withdrawable (the toggleAutoStake escape is no longer needed). (commit 5e10d0a)

H002 · BUSINESS LOGIC (SC02-LOGICERRORS)

HIGH

Founder's 90% Retirement Fees Permanently Locked in withdrawPendingFees

✓ FIXED

DESCRIPTION

withdrawPendingFees pays two distinct buckets pendingFounderWithdrawals (90%) and pendingTaxFeeWithdrawals (10%) in sequence, then runs a final "nothing to withdraw" guard. After the founder block pays and zeroes its bucket, the local amount variable is reassigned to the tax bucket (which is 0 for a wallet that only holds founder fees), and the final guard if (amount == 0 && pendingFounderWithdrawals[msg.sender] == 0 && pendingTaxFeeWithdrawals[msg.sender] == 0) revert NoPendingFees(); re-reads the now-zeroed mappings, evaluates all-true, and reverts rolling back the founder's own valid transfer. Since founderWallet and taxFeeWallet are distinct addresses, the founder wallet always has only founder fees and therefore always hits this path.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/CarbonFi_Retire.sol#L197-L214
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/CarbonFi_Retire.sol#L182-L192

BUSINESS IMPACT

90% of all retirement-fee revenue (the founder share) is permanently unwithdrawable; every founder withdrawal reverts. The condition is only avoided if founderWallet == taxFeeWallet, which is not enforced and not the intended configuration. Rotating the founder's wallet does not help the new wallet hit the identical path.

RECOMMENDED NEXT STEP

Capture both pending amounts in local variables before zeroing, and gate the revert on those pre-zero values.

RETEST

Fixed. withdrawPendingFees now reads founderAmount and taxAmount up front, checks totalPending == 0 at the start, and pays each bucket independently with no trailing zeroed-mapping guard (CarbonFi_Retire.sol). A founder-only wallet withdraws correctly. (commit 5e10d0a)

M001 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

MEDIUM

Listing Fee Evasion via Unvalidated Seller-Declared totalCo2eTon

✓ FIXED

DESCRIPTION

listItem computes the CAFI listing fee solely from a seller-supplied parameter: listingFeeCAFI = (totalCo2eTon * listingFeePerTon) / 1e18. totalCo2eTon is only floor-checked (>= 0.01e18) and is never linked to the listed amount or to the NFT's true on-chain tonnage. By design 1 CarbonNFT unit = 1 ton, so the real CO2e of a listing equals amount; however the contract never reads amount for the fee. A seller can therefore list an arbitrarily large quantity (e.g. 10,000 units / 10,000 tons) while declaring totalCo2eTon = 0.01e18 and pay only the floor fee. Because the listing fee is retained by the platform when the item sells, every seller is incentivized to declare the minimum, fully defeating the per-ton fee model. The fabricated totalCo2eTon is also stored and emitted, feeding incorrect CO2e figures to any off-chain UI/indexer.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L143
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L158

BUSINESS IMPACT

Sellers can pay only the 0.01-ton floor fee on arbitrarily large listings, systematically under-collecting protocol listing-fee revenue by a factor of realTons / 0.01, and the fabricated totalCo2eTon that is stored and emitted misinforms buyers and off-chain indexers. There is no direct theft of user funds.

RECOMMENDED NEXT STEP

Do not trust the seller-declared CO2e value. Because one NFT unit equals one ton, derive the listing fee from the listed quantity itself, or read the true tonnage from the CarbonNFT contract on-chain, and remove the unvalidated declared-tonnage parameter optionally enforcing that any declared CO2e is consistent with the listed quantity.

RETEST

Fixed. The seller-supplied totalCo2eTon parameter was removed from listItem; the listing fee is now derived from the listed quantity totalCo2eTon = amount * TOKEN_PER_TON then listingFeeCAFI = (totalCo2eTon * listingFeePerTon) / 1e18 (Carbonfi_Marketplace.sol). Sellers can no longer under-declare tonnage on a large listing.

M002 · BUSINESS LOGIC (SC02-LOGICERRORS)

Signature Orders Cannot Be Cancelled and Have Unbounded Expiry

MEDIUM

✓ FIXED

DESCRIPTION

A signed sell order in CarbonDEX remains valid until it is fully filled (filledAmount[orderHash] == order.amount) or until order.expiry passes. There is no mechanism to invalidate an order earlier: no cancelOrder function exists (although the OrderCancelled event and NotOrderOwner error are declared), the per-maker nonce mapping is never read or incremented, and MAX_ORDER_DURATION is never enforced so order.expiry may be set arbitrarily far in the future. Consequently a maker who signs an order cannot retract it. If the market price moves against a previously signed order, any taker can call matchOrderWithSignature with the old order/signature and force-fill it at the stale price, as long as the maker still holds the NFT and still has the contract approved as ERC1155 operator. The maker's only on-chain defense is setApprovalForAll(dex, false), which is a global kill-switch cancelling all of the maker's orders at once and which an old order will outlive once approval is re-enabled for a new order.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L129
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L78
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L72

BUSINESS IMPACT

A maker cannot retract a single signed order, so stale orders stay fillable by any taker at outdated prices and the only defense revoking operator approval cancels all of the maker's orders at once; combined with unbounded expiry, an order can be force-filled arbitrarily far into the future. No funds are stolen, but the maker is exposed to value loss on price moves.

RECOMMENDED NEXT STEP

Implement order revocation, either through a dedicated cancel function that marks an order fully filled. Additionally, enforce the existing maximum-order-duration bound on each order's expiry at fill time so orders cannot remain valid arbitrarily far into the future.

RETEST

Fixed. cancelOrder(Order) was added (CarbonDEX.sol) and is enforced in the matcher via cancelledOrders[orderHash]; the max-duration bound is now enforced at fill if (order.expiry > block.timestamp + MAX_ORDER_DURATION) revert MaxExpiryExceeded(). Both original gaps are closed.

M003 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

FIXED

Reward-Pool Double-Counting and Incorrect Require Cause Claim DoS / Stuck Rewards

DESCRIPTION

compoundReward parks the accrued reward via info.compoundedAmount += reward; rewardPoolBalance -= reward; already removing it from the pool. Later, claimReward's auto-compounding branch computes totalToTransfer = info.compoundedAmount + reward and executes rewardPoolBalance -= totalToTransfer, subtracting compoundedAmount a second time, so rewardPoolBalance drifts below the actual rewardToken held. Additionally, the guard checks require(rewardPoolBalance >= reward) while the branch subtracts compoundedAmount + reward; when reward <= rewardPoolBalance < compoundedAmount + reward, the require passes and the subtraction underflow-reverts. An auto-staker with a sizeable compoundedAmount against the drifted-low pool can then never claim, and once the gate value is depleted, claims revert for other stakers too even though real reward funds remain. pendingRewardWithdrawals is credited correctly, so this is a DoS / accounting-corruption issue, not an over-withdrawal.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L250-L255
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L184
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L188-L196

BUSINESS IMPACT

rewardPoolBalance is double-debited on every compounded claim, so its accounting drifts progressively below real holdings and demands escalating owner top-ups rather than a one-time refill. The artificially deflated balance then blocks both claiming (claimReward underflow-reverts at) and compounding (gated at require(rewardPoolBalance >= reward)) even while the contract physically holds enough rewardToken, stranding rewards until the owner refills the pool. There is no theft or over-payment; the defect is accounting corruption and denial of service.

RECOMMENDED NEXT STEP

Ensure the reward pool is debited only once for each compounded reward at compound time or at claim time, but not both and make the solvency check guard the full amount actually being moved out rather than only the newly-accrued reward. The cleanest approach treats compounded rewards as a deferred liability that is debited and solvency-checked a single time at claim, which also removes the unnecessary compound-time liquidity gate so that re-investing already-earned rewards never freezes.

RETEST

Fixed. The pool is now debited exactly once per reward compoundReward debits reward (Staking.sol); claimReward/unstake debit only reward, not reward + compounded (Staking.sol), since the compounded portion was already debited at compound time. No more drift / premature DoS. (commit 5e10d0a)

M004 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

FIXED

Compounded Rewards Permanently Forfeited When Auto-Staking Is Toggled Off Before Claim

DESCRIPTION

claimReward pays out a stake's accumulated compoundedAmount ONLY inside the branch guarded by if (info.autoStaking && info.compoundedAmount > 0). The info.autoStaking term makes the payout of already-earned compounded rewards depend on the current auto-staking toggle, which has nothing to do with whether those rewards are owed. The else branch credits only the freshly-accrued reward and ignores compoundedAmount entirely.
A user can reach the state (autoStaking == false, compoundedAmount > 0) through a natural, contract-permitted sequence:

- Stake with auto-staking enabled and call compoundReward ≥ 1 time compoundReward does info.compoundedAmount += reward (L250). (Note: compoundedAmount can ONLY become > 0 via this path, so auto-staking was necessarily true at some point.)
- Call toggleAutoStake(stakeIndex) unguarded except !claimed flipping info.autoStaking false while compoundedAmount stays > 0.
- Call claimReward after unlock. The condition at is now false execution falls to the else branch only reward is credited; compoundedAmount is never added to pendingRewardWithdrawals. info.claimed was set true at before the branch, and both claimReward and compoundReward gate on !claimed, so the stake can never be re-entered. The compounded rewards are permanently forfeited.

This is a realistic flow, not a contrived one: a user who has been compounding and now wants to cash out would intuitively (a) toggle auto-staking off to stop further compounding, then (b) claim and that exact two-step sequence silently destroys every compounded reward, with no revert and no warning.
The forfeited tokens are also orphaned in the contract: compoundReward already did rewardPoolBalance -= reward without transferring tokens out, so the underlying reward tokens sit in the contract balance but are no longer tracked by rewardPoolBalance and are never credited to any pendingRewardWithdrawals. Staking.sol has no emergency/arbitrary-token sweep, so they are unrecoverable by anyone, a permanent loss to the user and a permanent lock in the contract.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L186-L199
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L222-L231
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L250-L255

BUSINESS IMPACT

A user who toggles auto-staking off and then claims permanently and unrecoverably loses every already-earned compounded reward, and the forfeited tokens are stranded in the contract (untracked by rewardPoolBalance, no sweep) and permanently locked. The loss is bounded to the compounded portion rather than principal, is user-triggered, and has no cross-user or attacker vector.

RECOMMENDED NEXT STEP

Pay out the accumulated compounded rewards whenever they are non-zero, independent of the current auto-staking toggle; the auto-staking term must be removed from the claim branch, since those rewards are owed regardless of whether auto-staking is still enabled. And ensure that the fee for auto-compounding is calculated or better approach would be to disallow toggling on to off when compounded rewards are greater than zero.

RETEST

Fixed. toggleAutoStake now settles the accrued reward at the current mode before flipping the flag (Staking.sol): it computes the accrued reward, credits it to info.compoundedAmount, advances info.lastRewardClaimTime to block.timestamp, and debits rewardPoolBalance. The dormant post-unlock window can therefore no longer be re-priced at the uncapped auto rate, and the legitimately earned capped reward is preserved rather than erased exactly the recommended remediation. (commit 6fe5d98)

M005 · BUSINESS LOGIC (SC02-LOGICERRORS)

Escrowed NFT and Locked CAFI Trapped When a Credit Expires Mid-Listing

MEDIUM

✓ FIXED

DESCRIPTION

The L003 remediation added a notExpired(id) check to every CarbonNFT.safeTransferFrom, including transfers out of contract escrow. The marketplace escrows a listed NFT in listItem; if the credit's end timestamp passes while it is listed, every exit path reverts with CreditExpired buyItem, cancelListing (so the seller cannot retrieve their own NFT), and even emergencyWithdrawNFT. Both the escrowed NFT and the seller's locked CAFI listing fee become permanently unrecoverable.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Mint_NFT.sol#L122-L128
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Mint_NFT.sol#L170-L178
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Carbonfi_Marketplace.sol#L193
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Carbonfi_Marketplace.sol#L210
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Carbonfi_Marketplace.sol#L255-L259

BUSINESS IMPACT

Any listing that crosses its credit's expiry permanently traps the NFT and the seller's locked CAFI, with no recovery path for the seller or the owner.

RECOMMENDED NEXT STEP

Enforce expiry at acquisition/list/buy time rather than on all transfers, or exempt escrow-return and burn paths (e.g. from == marketplace or to == 0xdead) from the expiry block so cancelListing/emergencyWithdrawNFT can always return escrow.

RETEST

Fixed. buyItem now rejects expired credits: a private _isExpired(tokenId) helper staticcalls CarbonNFT.isExpired(uint256), and buyItem reverts with CreditExpired() before any state change or transfer (Carbonfi_Marketplace.sol).

M006 · BUSINESS LOGIC (SC02-LOGICERRORS)

Listing Fee Divided by 1e18 Approximately 1e18× Too Small

MEDIUM

✓ FIXED

DESCRIPTION

The M-01 remediation removed the seller-supplied totalCo2eTon and auto-derives the listing fee, but the new formula divides by 1e18: listingFeeCAFI = (totalCo2eTon * listingFeePerTon) / 1e18. Here totalCo2eTon = amount * TOKEN_PER_TON is a raw ton count (not 18-decimal) and listingFeePerTon is already an 18-decimal CAFI amount, so the extra /1e18 makes the collected fee approximately 1e18× too small. For a fee of 10 CAFI/ton (10e18) and 5 tons, the intended fee is 50 CAFI (50e18) but the code computes 50 wei. For small inputs the product truncates to 0 and revert ListingFeeTooSmall() blocks the listing entirely.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Carbonfi_Marketplace.sol#L146-L148

BUSINESS IMPACT

The protocol collects essentially no listing fee (the fee rounds to dust), or listItem reverts (DoS) for small amounts / low listingFeePerTon. Either way the M-01 fee mechanism is broken.

RECOMMENDED NEXT STEP

Remove the /1e18 (the operands are already correctly scaled): listingFeeCAFI = totalCo2eTon * listingFeePerTon;. If fractional CAFI-per-ton precision is intended, document the scaling of listingFeePerTon and only keep /1e18 where totalCo2eTon is itself 18-decimal scaled.

RETEST

Fixed. The /1e18 was removed listingFeeCAFI = totalCo2eTon * listingFeePerTon (Carbonfi_Marketplace.sol) so the configured per-ton fee is collected at full scale. (commit 5e10d0a)

M007 · BUSINESS LOGIC (SC02-LOGICERRORS)

Partial Fills Unfillable Maker Balance Checked Against Full order.amount

MEDIUM

✓ FIXED

DESCRIPTION

The partial-fill path checks the maker's NFT balance against the full original order.amount rather than the amount being filled or the remaining amount. Each fill transfers NFTs out of the maker, so after the first partial fill the maker's balance drops below order.amount and every subsequent fill reverts InsufficientListing, even though the maker still holds enough for the remainder. A partially-filled order can therefore never be completed directly defeating the advertised partial-fill feature.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/CarbonDEX.sol#L178

BUSINESS IMPACT

No direct fund loss, but order liquidity is stranded: any order filled in pieces becomes permanently stuck after the first fill, breaking a core advertised feature.

RECOMMENDED NEXT STEP

Check against the amount actually required: if (CarbonNFT.balanceOf(order.maker, order.tokenId) < amountToFill) revert InsufficientListing(); (or against remaining).

RETEST

Fixed. The maker-balance check now compares against amountToFill instead of order.amount (CarbonDEX.sol:172), so partially-filled orders remain fillable for their remainder. (commit 5e10d0a)

M008 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

FIXED

Compounded Rewards Permanently Forfeited When Unstaking a Matured Stake With Zero Fresh Reward

DESCRIPTION

The reward-split rewrite re-introduces the forfeiture class in a new location: unstake. unstake pays a stake's accumulated compoundedAmount ONLY inside the branch guarded by if (totalReward > 0) (Staking.sol, where paidReward = totalPayout and totalPayout = baseReward + netAuto + info.compoundedAmount). But the matching ledger write totalCompounded -= info.compoundedAmount (Staking.sol) and info.isActive = false both run UNCONDITIONALLY, before that guard. So when a stake is unstaked while totalReward == 0 but info.compoundedAmount > 0, the reward-payout block is skipped entirely: the user receives their principal but NOTHING for their compounded rewards, while the accounting still drops them. The compounded tokens are then orphaned in the contract no longer reserved by totalCompounded and never credited to any pendingRewardWithdrawals so the user loses them permanently and only the owner can later recover them via emergencyWithdrawExcessRewardToken (they show up as an unreserved surplus).
totalReward == 0 at unstake (which already requires block.timestamp >= unlockTime) is exactly the state lastRewardClaimTime >= unlockTime on a non-auto stake (post-unlock, no fresh accrual), and it is reached through a natural, contract-permitted sequence:

- Stake with auto-staking enabled (info.compoundedAmount can ONLY become > 0 via a settlement, so auto-staking was necessarily on at some point).
- Let the stake mature (block.timestamp >= unlockTime).
- Call toggleAutoStake(stakeIndex) it settles the accrued reward into info.compoundedAmount, sets info.lastRewardClaimTime = block.timestamp (which is now >= unlockTime), and flips info.autoStaking □ false, but does NOT advance unlockTime (Staking.sol). For the now-non-auto stake, reward only accrues over [lastRewardClaimTime, unlockTime], an empty window (lastRewardClaimTime >= unlockTime). totalReward is therefore 0 from this point on, permanently.
- Call unstake(stakeIndex). With totalReward == 0 and compoundedAmount > 0, the compounded rewards are silently destroyed.

This is the same intuitive flow M004 flagged a user who has been auto-compounding toggles auto-staking off to stop compounding and then exits except the retest-3 fix relocated the forfeiture from claimReward into unstake. That it is an oversight rather than intended is proven by claimReward, which handles the identical state correctly: its guard is require(totalReward > 0 || info.compoundedAmount > 0) (Staking.sol) and it credits totalPayout (including compoundedAmount) to pending UNCONDITIONALLY (Staking.sol). Only unstake gates the compounded payout behind totalReward > 0. The safe-but-non-obvious workaround is to call claimReward before unstake; calling unstake directly the natural one-shot exit burns the compounded rewards.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/763b7119bba69269f99dccf330ae92c83fa1c7e4/Staking.sol#L238-L261
- https://github.com/aresa1801/cafi-contract/blob/763b7119bba69269f99dccf330ae92c83fa1c7e4/Staking.sol#L229-L231
- https://github.com/aresa1801/cafi-contract/blob/763b7119bba69269f99dccf330ae92c83fa1c7e4/Staking.sol#L292-L306

BUSINESS IMPACT

A user who toggles auto-staking off on a matured stake and then unstakes (instead of claiming first) permanently and unrecoverably loses every already-earned compounded reward; the forfeited tokens are stranded in the contract (untracked by totalCompounded, no pending credit) and recoverable only by the owner via emergencyWithdrawExcessRewardToken. The loss is bounded to the compounded portion rather than principal, is user-triggered with no cross-user or attacker vector, but it is silent (no revert, no warning) and irreversible once unstake is called.

RECOMMENDED NEXT STEP

Pay out info.compoundedAmount whenever it is non-zero, independent of totalReward, mirroring claimReward. Minimal fix: change the guard at Staking.sol:242 from if (totalReward > 0) to if (totalReward > 0 || info.compoundedAmount > 0); when totalReward == 0 the rewardPoolBalance >= totalReward branch holds trivially and paidReward = totalPayout = compoundedAmount is transferred (compounded is backed by tokens already held, so it pays out even with an empty pool). A cleaner alternative is to settle info.compoundedAmount separately and always in full, applying the pool-availability (L009) partial logic only to the fresh totalReward.

RETEST

Fixed. Unstake now gates the reward-payout block on if (totalReward > 0 || info.compoundedAmount > 0) (Staking.sol) instead of if (totalReward > 0). When totalReward == 0 and info.compoundedAmount > 0, execution now enters the block, the rewardPoolBalance >= totalReward branch holds trivially (totalReward is 0), and paidReward = totalPayout = compoundedAmount is transferred to the user (the compounded principal is backed by tokens already held, so it pays out even with an empty reward pool). The remainder of unstake is unchanged.

L001 · MISSING FUNCTIONALITY (SCWE-006)

Missing Handling of Fees on Transfer in ERC20 Token Transfers

LOW

✓ FIXED

DESCRIPTION

The contract contains a vulnerability related to transferring ERC20 tokens without considering the possibility of fees charged on transfer. Some ERC20 tokens implement a fee mechanism, where a certain percentage of tokens is deducted as a fee during each transfer. However, the contract does not account for this possibility when transferring tokens using the safeTransferFrom/TransferFrom function.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L106
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L268

BUSINESS IMPACT

Failure to account for transfer fees can lead to accounting errors and financial losses. When tokens with transfer fees are transferred, the actual received amount may be less than expected due to the deduction of fees. As a result, the contract's internal accounting and balance tracking may become inaccurate, leading to discrepancies in token balances and potential financial losses for users.

RECOMMENDED NEXT STEP

To address this vulnerability it is recommended to add a mechanism to calculate the balance of the contract before and after the transfer is completed.

RETEST

Fixed. Stake and addRewardPoolFunds now measure the balance delta around the pull and reject non-1:1 transfers require(actualReceived == amount, "fee-on-transfer not supported") (Staking.sol:127-130, 278-281). The chosen approach (reject fee-on-transfer tokens) prevents the accounting drift at the cited transfer sites.

L002 · BEST PRACTICE

Single-Step Ownership Transfer (Ownable instead of Ownable2Step)

LOW

✓ FIXED

DESCRIPTION

CarbonFi and CarbonFi_Retire both inherit OpenZeppelin Ownable, whose transferOwnership transfers ownership in a single step with no acceptance from the incoming owner. A transfer to a mistyped or uncontrolled address irreversibly forfeits all admin capability (pause, exclusion management, owner burn). The rest of the suite uses Ownable2Step, so this is also an inconsistency. Team note: acknowledged / in-contract note added.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi.sol#L14
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L15

BUSINESS IMPACT

An accidental or incorrect single-step transferOwnership permanently bricks all administrative control of the token, with no acceptance step to catch a mistyped or uncontrolled address.

RECOMMENDED NEXT STEP

Use Ownable2Step (two-step accept) for ownership transfers, consistent with the other contracts in the suite.

RETEST

Fixed. CarbonFi (CarbonFi.sol) and CarbonFiRetirement (CarbonFi_Retire.sol) both inherit Ownable2Step now, consistent with the rest of the suite; ownership transfer requires explicit acceptance by the incoming owner.

L003 · BUSINESS LOGIC (SC02-LOGICERRORS)

Carbon Credit Expiry (end) Stored but Never Enforced

LOW

✓ FIXED

DESCRIPTION

At mint, CarbonNFT records each credit's validity window as Project.end = start + durationDays * 1 days (Mint_NFT.sol), with durationDays capped at MAX_DURATION_DAYS (~10 years) strongly implying credits are meant to be valid only for a finite period. However, no contract ever reads or enforces end.

The most consequential gap is on the retirement path. CarbonFiRetirement.retireNFT gates retirement only on NFT balance, operator approval, and CAFI fee; it performs no temporal/expiry check and stamps the certificate with the current block.timestamp regardless of the credit's end. Because there is also no holding period or lockup, a credit can be retired (a) immediately after mint, or (b) at any future time including long after Project.end has passed and will mint a valid-looking offset certificate indistinguishable from a fresh credit. CarbonNFT itself and Carbonfi_Marketplace likewise ignore the end. The on-chain expiry is therefore dead metadata, and the protocol cannot prevent expired credits from being claimed as valid offsets.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Mint_NFT.sol#L221
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Mint_NFT.sol#L73
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Mint_NFT.sol#L284
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L87

BUSINESS IMPACT

Expired carbon credits remain fully usable tradable, collateralizable, and, most importantly, retirable and retiring one yields an offset certificate indistinguishable from a valid credit, undermining the integrity of the protocol's core product and permitting the stale-credit claims the validity window was presumably meant to prevent. The stored durationDays/end is therefore misleading, advertising an on-chain validity guarantee that does not exist.

RECOMMENDED NEXT STEP

Decide the intended semantics and make the code match. If credits are meant to expire, enforce the expiry on the retirement path (and optionally on listing and lending) by having the retirement function read the credit's stored end date and reject stale credits

RETEST

Fixed. A notExpired modifier (Mint_NFT.sol) now gates safeTransferFrom and retire; safeBatchTransferFrom/batchRetire carry inline expiry checks; an isExpired(tokenId) view was added. Expired credits can no longer be transferred or retired.

L004 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

Non-Unique Retirement Certificate ID Allows Record Overwrite

✓ FIXED

DESCRIPTION

retireNFT derives the certificate ID deterministically as "CRT-" + chainId + "-" + hex(msg.sender) + "-" + tokenId + "-" + block.timestamp. For a fixed (chainId, msg.sender, tokenId) the only varying component is block.timestamp and block.timestamp is a block-header field, identical for every transaction in a block (the EVM has no per-transaction timestamp). The collision is therefore deterministically reproducible, not a timing race:

- Single-transaction vector (100% reproducible, no timing luck): retireNFT is external with no EOA restriction, and nonReentrant blocks only nested re-entry sequential calls in one tx are allowed (the guard releases when each call returns). So a contract (or any multicall/batcher) holding a carbon credit can call retireNFT(tokenId, x) then retireNFT(tokenId, y) in a single transaction. Within one tx all four ID components are provably constant chainId, msg.sender, tokenId, and the block-constant block.timestamp so the second call produces an identical certificateId and overwrites the first.
- Two-transaction / same-block vector: an EOA broadcasting two retireNFT txs that land in the same block shares one block.timestamp and collides identically.
- Cross-block widening (target chains): block.timestamp has 1-second resolution and is only required to be non-decreasing on relaxed chains. On Arbitrum and fast/sub-second BNB blocks (both explicit deployment targets), even retirements in different blocks can share a timestamp.

On collision the second retirement OVERWRITES retirementCertificates[certificateId]: both burns and both fees still occur and totalRetiredByTokenId accumulates correctly, but only one certificate record survives and its amount field reflects only the last retirement. The collision is self-only (the ID embeds the sender's address), so there is no cross-user forging/griefing and no way to over-count.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L94
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L123

BUSINESS IMPACT

Retiring the same credit twice in one block overwrites the first certificate, leaving a single record that covers two burns but stores only the last amount, corrupting the retirement-certificate registry and feeding ambiguous or overwritten data to any off-chain certificate generation keyed on the ID. There is no fund or NFT loss, the aggregate retired count stays correct, and there is no cross-user impact.

RECOMMENDED NEXT STEP

Make each certificate ID unique independently of the block timestamp by incorporating a strictly increasing per-user counter (a global counter or a per-user-per-token nonce works equally well) so that two retirements by the same address can never collide regardless of timestamp, same-block inclusion, or chain timestamp semantics. Avoid relying solely on a "record already exists" revert, since that would reject legitimate same-block retirements rather than disambiguate them.

RETEST

Fixed. The certificate ID now incorporates a strictly-increasing per-user counter ++userCertificateNonce[msg.sender] (CarbonFi_Retire.sol), so same-block / same-tx retirements by one address can no longer collide (commit 5e10d0a).

L005 · FLOATING PRAGMA (SCWE-060)

Floating and Outdated Pragma

LOW

ACCEPTED RISK

DESCRIPTION

Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.
The contract allowed floating or unlocked pragma to be used, i.e., >= 0.8.24. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi.sol#L2
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L2
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L2
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L2
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L2
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Mint_NFT.sol#L2

BUSINESS IMPACT

If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.
Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.
The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.34 pragma version
Reference: https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/

RETEST

ACKNOWLEDGED. All six files remain locked to pragma solidity 0.8.24; (floating risk resolved); the recommended upgrade to the latest 0.8.34 was not adopted. (commit 5e10d0a).

L006 · CHECKS-EFFECTS-INTERACTIONS

LOW

CEI Violation in buyItem and cancelListing (NFT Transfer Before State Update)

✓ FIXED

DESCRIPTION

Both buyItem and cancelListing transfer the escrowed ERC1155 NFT to msg.sender BEFORE updating contract state. In buyItem the NFT is sent before totalLockedCAFI, listing.amount, and listing.listingFeePaid are decremented. In cancelListing the NFT is sent before the CAFI refund, totalLockedCAFI decrement, and delete . ERC1155 safeTransferFrom invokes onERC1155Received on the recipient; were the nonReentrant guard absent, a re-entrant cancelListing would observe the not-yet-deleted listing and could transfer listing.amount NFTs again draining other sellers' escrowed tokens for the same tokenId, since escrow is shared per tokenId and refund the CAFI fee twice. Currently this is blocked because listItem/buyItem/cancelListing all share OpenZeppelin's single nonReentrant status flag, leaving no unguarded state-changing entry point reachable during the callback.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L194-L204
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L214-L222

BUSINESS IMPACT

The issue is not exploitable in the current code thanks to the shared nonReentrant guard, but the ordering is fragile: removing the guard, adding a future non-guarded entry point, or introducing a delegatecall path would enable double-refund of CAFI and theft of other sellers' escrowed NFTs.

RECOMMENDED NEXT STEP

Apply strict Checks-Effects-Interactions: update all state and delete the listing BEFORE the external NFT transfer. Keep nonReentrant as defense-in-depth.

RETEST

Fixed. Both functions now apply effects before interactions: buyItem decrements listing.amount/listingFeePaid/totalLockedCAFI and deletes the slot before any token transfer (Carbonfi_Marketplace.sol); cancelListing updates totalLockedCAFI and deletes the listing before the NFT/CAFI transfers. nonReentrant retained as defense-in-depth.

L007

Missing events in important functions

LOW

✓ FIXED

DESCRIPTION

VulnerabilityType
Missing Best Practices
Events are inheritable members of contracts. When you call them, they cause the arguments to be stored in the transaction's log a special data structure in the blockchain. These logs are associated with the address of the contract which can then be used by developers and auditors to keep track of the transactions.
The contract was found to be missing these events on certain critical functions which would make it difficult or impossible to track these transactions off-chain.
AffectedCode

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L188
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L196
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L258

BUSINESS IMPACT

Events are used to track the transactions off-chain and missing these events on critical functions makes it difficult to audit these logs if they're needed at a later stage.

RECOMMENDED NEXT STEP

Consider emitting events for important functions to keep track of them.

RETEST

Fixed. The marketplace emergency movers now emit events emergencyWithdrawERC20->EmergencyWithdrawERC20, emergencyWithdrawNFT->EmergencyWithdrawNFT (Carbonfi_Marketplace.sol); CarbonFiRetirement.emergencyWithdraw also emits EmergencyWithdraw (commit 5e10d0a).

L008 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

Buy Fee on buyItem Is Borne by the Seller and Stacks on the Seller's Listing Fee

ACCEPTED RISK

DESCRIPTION

buyItem charges a buyFeeBps fee on each purchase, but the fee is deducted from the seller's USDT proceeds rather than added on top of the buyer's payment: the buyer transfers exactly pricePerItem * amount (split into buyFeeUSDT sent to feeWallet and sellerAmountUSDT sent to the seller), so the seller receives totalPriceUSDT − buyFeeUSDT. Despite the name "buyFee," the seller bears it. Separately, the seller already paid a CAFI listing fee at listItem, so a completed sale charges the seller twice once in CAFI at listing and again in USDT at sale while the buyer pays only the list price. Whether charging both fees is intended, and whether the buy fee is meant to fall on the seller or be added on top for the buyer, is a design-intent question for the team (raised as a clarification question).

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L23
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L179
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L188-L189
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L191-L192
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L158

BUSINESS IMPACT

There is no security impact; this is a fee-incidence and. As written, the seller is charged on both sides of a trade a CAFI listing fee plus a USDT buy fee taken from proceeds while the misleading "buyFee" name implies the buyer pays it, which can confuse integrators and sellers about the true economics of a sale.

RECOMMENDED NEXT STEP

Confirm the intended fee model. If the buy fee is meant to fall on the buyer, charge it on top of the list price so the buyer pays price plus fee and the seller receives the full price; if only a single fee is intended, drop either the listing fee or the buy fee. In all cases, rename the fee to reflect who actually bears it so the contract's economics are not misleading.

RETEST

ACKNOWLEDGED.

L009 · BUSINESS LOGIC (SC02-LOGICERRORS)

Unstake Couples Principal Recovery to Reward-Pool Solvency

LOW

✓ FIXED

DESCRIPTION

unstake reverts the entire call including the principal transfer when the reward pool cannot cover the accrued reward: require(rewardPoolBalance >= totalPayout, "insufficient reward pool") gates the subsequent stakingToken.safeTransfer(msg.sender, principal). Principal recovery therefore depends on the owner keeping the reward pool funded, even though rewards are never reserved per-stake at stake time. The condition is reachable through ordinary operation (high APY on a large stake) and is amplified by C002 and M003, which can push the pool short.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L151-L180

BUSINESS IMPACT

A staker's principal is stranded whenever the reward pool is short, until the owner voluntarily tops it up with no obligation to do so.

RECOMMENDED NEXT STEP

Always return principal; pay rewards only up to the available rewardPoolBalance (route any shortfall to pendingRewardWithdrawals, or pay min(totalPayout, rewardPoolBalance)) instead of reverting the whole call.

RETEST

Fixed. unstake transfers principal unconditionally first (Staking.sol) and routes any reward-pool shortfall to pendingRewardWithdrawals instead of reverting, so principal recovery no longer depends on pool solvency. (commit 5e10d0a)

L010 · BUSINESS LOGIC (SC02-LOGICERRORS)

Pause Not Enforced on NFT Transfers

LOW

✓ FIXED

DESCRIPTION

mint and retire carry whenNotPaused, but the safeTransferFrom and safeBatchTransferFrom overrides do not, and OpenZeppelin 4.9.5's _beforeTokenTransfer hook is empty. Holders can therefore continue to move (and launder) credits while the contract is "paused," defeating the emergency stop.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Mint_NFT.sol#L170-L178
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Mint_NFT.sol#L180-L194

BUSINESS IMPACT

The pause mechanism does not actually halt credit transfers, weakening the intended emergency response.

RECOMMENDED NEXT STEP

Override _beforeTokenTransfer with the whenNotPaused modifier, which uniformly covers single transfers, batch transfers, mints, and burns.

RETEST

Fixed. _beforeTokenTransfer now carries whenNotPaused (Mint_NFT.sol), so a pause halts all NFT transfers (and mint/burn). (commit 5e10d0a)

L011 · BUSINESS LOGIC (SC02-LOGICERRORS)

Wallet Rotation Strands Already-Accrued Pending Fees

LOW

✓ FIXED

DESCRIPTION

Pending fee balances are keyed by the wallet address at retire time. setFounderWallet and setTaxFeeWallet only repoint the state variable; they do not migrate the already-accrued balances. If a wallet is rotated because its key was lost or compromised, the fees accrued under the old address become permanently unclaimable (and the CAFI cannot be swept emergencyWithdraw blocks CAFI).

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/CarbonFi_Retire.sol#L232-L245

BUSINESS IMPACT

Accrued founder/tax fees under a rotated-away wallet are stranded with no recovery path.

RECOMMENDED NEXT STEP

Migrate the pending balance to the new wallet on change, or add an owner-only reassignPending(old, new) function.

RETEST

Fixed. setFounderWallet/setTaxFeeWallet migrate accrued pending fees to the new wallet on rotation, and an owner-only migratePendingFees(from,to,amount) was added (CarbonFi_Retire.sol). (commit 5e10d0a)

L012 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

MAX_STAKES_PER_USER Counts Historical Stakes Permanent Staking Lockout

✓ FIXED

DESCRIPTION

unstake only flips info.isActive = false; it never removes the entry from the user's stakes array, which is append-only. stake caps on stakes[msg.sender].length < MAX_STAKES_PER_USER (20), counting inactive (withdrawn) slots. After 20 lifetime stakes a user can never stake again, even with zero active positions.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L123
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L151-L180

BUSINESS IMPACT

A user is permanently locked out of staking after 20 lifetime deposits, regardless of how many are still active. Self-inflicted, no fund loss.

RECOMMENDED NEXT STEP

Track an activeStakeCount (incremented on stake, decremented on unstake) and enforce the cap against it, or compact the array (swap-and-pop) on unstake.

RETEST

Fixed. A dedicated activeStakeCount (incremented on stake, decremented on unstake) now gates the cap instead of array length (Staking.sol), so inactive slots no longer block new stakes. (commit 5e10d0a)

L013 · BUSINESS LOGIC (SC02-LOGICERRORS)

Compound Fee Over-Applied and Trivially Evadable

LOW

✓ FIXED

DESCRIPTION

The compoundFeeBps fee is charged in exactly one place claimReward when autoStaking && block.timestamp >= unlockTime and there it is applied to the entire payout (reward + compoundedAmount), over-taxing base rewards. Meanwhile compoundReward (the named action) charges nothing, and unstake pays reward + compounded with no fee. A user can therefore avoid the fee entirely by exiting via unstake, by harvesting through compoundReward, or by calling toggleAutoStake to set autoStaking = false before claimReward. Net compound-fee revenue is effectively zero, while the few users who route through claimReward are over-charged.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L199-L203
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L229-L252
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L151-L180
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L220-L226

BUSINESS IMPACT

The protocol's intended compound-fee revenue is essentially uncollectable, and the few users who do pay are over-charged. No theft of user funds.

RECOMMENDED NEXT STEP

Charge the fee on the compounded portion at compound time and apply it consistently across exit paths .

RETEST

Fixed. The compound fee is now charged inside compoundReward on the amount being compounded (Staking.sol), and claimReward charges no fee so the fee is consistent, unavoidable when compounding, and no longer over-applied. (commit 5e10d0a)

L014 · ACCESS CONTROL (SCWE-016)

LOW

emergencyWithdrawExcessRewardToken Can Sweep User-Owed and Staked Tokens

✓ FIXED

DESCRIPTION

emergencyWithdrawExcessRewardToken computes withdrawable "excess" as rewardToken.balanceOf(address(this)) - rewardPoolBalance. This omits two reserved liabilities from the reserve calculation. First, when a reward is claimed, claimReward debits rewardPoolBalance but the tokens remain in the contract as pendingRewardWithdrawals until the user pulls them so already-owed rewards count as "excess." Second, and more seriously, in the canonical single-token deployment where stakingToken == rewardToken (stake CAFI to earn CAFI), balanceOf(address(this)) also includes the entire staked principal, so excess evaluates to totalStaked and the owner can withdraw all stakers' principal. This directly contradicts the "FIX M-01: cannot drain reserved pool" claim in the contract header.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L327-L335

BUSINESS IMPACT

Depending on configuration, the owner can withdraw reward tokens already owed to users (breaking their later withdrawRewards), or in the same-token deployment drain all staked principal. The function presented as a safe "excess only" withdrawal is not safe.

RECOMMENDED NEXT STEP

Track totalPendingWithdrawals and define reserved = rewardPoolBalance + totalPendingWithdrawals, adding + totalStaked when address(stakingToken) == address(rewardToken). Only balanceOf(address(this)) - reserved may be withdrawn as excess.

RETEST

Fixed. A totalCompounded aggregate was added (Staking.sol): incremented in compoundReward and in the new toggleAutoStake settlement , decremented in claimReward and unstake , and now included both in reserved and in the updateRewardToken liability guard . The owner can no longer sweep the reward tokens backing outstanding compounded balances. Verified totalCompounded stays equal to the sum of active compoundedAmount across all paths, with no double-decrement (claimReward zeroes compoundedAmount, so a subsequent unstake reads 0). (commit 6fe5d98)

L015 · BUSINESS LOGIC (SC02-LOGICERRORS)

Compound Fee Bypassable via toggleAutoStake Settlement

LOW

✓ FIXED

DESCRIPTION

toggleAutoStake settles the accrued reward before flipping the auto-staking flag. The settlement credited the full accrued reward to info.compoundedAmount with no compound fee, while compoundReward (the intended compounding path) charges compoundFeeBps (default 50 = 0.5%, max 1000 = 10%) on every compound. A user could therefore avoid the compound fee by calling toggleAutoStake to settle accrued rewards into compoundedAmount instead of compoundReward; toggleAutoStake has no unlock guard, so it could even be used before unlock. Because compoundedAmount is added to the reward-earning base (principal = amount + compoundedAmount), the fee-free settled rewards then earned further rewards on a larger base than the fee-charging path would produce. This partially re-opened L013 (compound-fee evasion).

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/6fe5d98bdfd9724a88819b96570109c236d03e72/Staking.sol#L267-L292
- https://github.com/aresa1801/cafi-contract/blob/6fe5d98bdfd9724a88819b96570109c236d03e72/Staking.sol#L298-L319

BUSINESS IMPACT

The protocol / feeReceiver lost compound-fee revenue, and users who settled via toggling compounded fee-free and on a slightly larger base than intended. No theft of user funds; impact scaled with compoundFeeBps (negligible at 0.5%, material at the 10% maximum).

RECOMMENDED NEXT STEP

Charge the compound fee on the toggleAutoStake settlement as well (mirroring compoundReward), or explicitly accept and document that settlement is fee-free; if kept fee-free, rate-limit how often a stake can be toggled so it cannot be used as a routine fee-free compound loop.

RETEST

Fixed. The client confirmed the model the base (pre-unlock) reward is intentionally fee-free and the auto (post-unlock) reward bears compoundFeeBps and the Staking.sol reward-split rewrite now charges compoundFeeBps on the auto portion inside toggleAutoStake (Staking.sol), identical to compoundReward (Staking.sol), claimReward (Staking.sol), and unstake (Staking.sol). The settlement can therefore no longer be used to avoid the compound fee; the fee is applied consistently to the same post-unlock auto portion across all four paths, closing the evasion. (commit 763b711)

I001

Dead Code

INFO

✓ FIXED

DESCRIPTION

VulnerabilityType
Code With No Effects (SCWE-062)
It is recommended to keep the production repository clean to prevent confusion and the introduction of vulnerabilities. The functions and parameters, contracts, and interfaces that are never used or called externally or from inside the contracts should be removed when the contract is deployed on the mainnet.
AffectedCode

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi.sol#L15
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L17
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L72
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L78

BUSINESS IMPACT

This does not impact the security aspect of the Smart contract but prevents confusion
when the code is sent to other developers or auditors to understand and implement.
This reduces the overall size of the contracts and also helps in saving gas.

RECOMMENDED NEXT STEP

If the library functions are not supposed to be used anywhere, consider removing them from the contract.

RETEST

Fixed. Both remaining items were removed the unused using Strings for uint256 directive in CarbonFiRetirement and the dead nonce mapping in CarbonDEX.

I002 · CODE QUALITY

removeNFTContract Leaves Stale Entries in nftContractList

INFO

✓ FIXED

DESCRIPTION

removeNFTContract sets supportedNFTContracts[nftContract] = false but never removes the address from the nftContractList array. Re-adding a previously removed contract passes the !supportedNFTContracts[...] check and pushes the address again, creating duplicates. getSupportedNFTContracts therefore returns stale (removed) and/or duplicated entries. Access control is unaffected because the mapping remains the source of truth for onlySupportedNFT.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L127
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L119

BUSINESS IMPACT

getSupportedNFTContracts returns inaccurate (stale and/or duplicated) data; there is no security impact on listing or buying, since the mapping remains the access-control source of truth.

RECOMMENDED NEXT STEP

Remove the address from nftContractList on removal (swap-and-pop), or otherwise keep the array reflecting only currently-supported contracts.

RETEST

Fixed. The whole multi-NFT registry (supportedNFTContracts, nftContractList, addNFTContract/removeNFTContract/getSupportedNFTContracts, onlySupportedNFT) was removed; the marketplace now binds to a single immutable carbonFiNFT (Carbonfi_Marketplace.sol). The stale-array condition no longer exists.

I003 · CODE QUALITY

Incomplete / Dead Code in CarbonDEX

INFO

✓ FIXED

DESCRIPTION

CarbonDEX ships with several unused or inconsistent elements:

- Dead feeWallet fees accrue to accumulatedFees and withdrawFees(to) sends them to an arbitrary owner-chosen address; feeWallet (set via setFeeParams) is never used to receive anything.
- Fee incidence contradicts the spec the header comment states "Fees collected from taker," but the taker pays exactly pricePerItem * amountToFill while the maker receives total - fee, so the fee is actually borne by the maker/seller. Because feeBps is mutable, a maker's net proceeds are not guaranteed by their signature.
- Buy orders are non-functional the order.isBuy branch always reverts (L172), with the original stream-of-consciousness design comments left in source.
- Stray USDT can be locked withdrawFees only transfers tracked accumulatedFees and emergencyWithdrawERC20 blocks USDT, so any USDT mistakenly sent to the contract cannot be recovered.
- No MIN_PRICE/MIN_AMOUNT bounds, and feeAmount = totalValue * feeBps / 10000 rounds to zero for dust totalValue.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L70
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L153-L155
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L172
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L199
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L208
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonDEX.sol#L216

BUSINESS IMPACT

No direct security impact; the issues affect clarity, maintainability, fee transparency, and the recoverability of mis-sent USDT.

RECOMMENDED NEXT STEP

Remove or wire up feeWallet; correct the fee-incidence documentation (or move the fee onto the taker if intended); remove or implement the buy-order path; allow recovery of non-fee USDT surplus; add minimum price/amount bounds if dust orders are a concern

RETEST

Fixed. feeWallet was removed (replaced by setFeeBps); rescueExcessUSDT() recovers stray USDT above accumulatedFees; and MIN_PRICE (1 USDT) + MIN_AMOUNT are enforced in matchOrderWithSignature, closing the dust-fee path (commit 5e10d0a).

I004 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

INFO

buyItem Missing amount > 0 Check (No-op Sales / Event Spam)

✓ FIXED

DESCRIPTION

buyItem is gated only by the validListing modifier (listing.amount >= amount and escrow >= amount), both of which pass when amount == 0. A zero-amount buy against a live listing performs only zero-value transfers but still emits ItemSold(..., amount=0, ...), letting anyone spam fake sale events against any listing and pollute off-chain indexers. (Against a non-existent listing it instead reverts via division-by-zero at platformEarnedCAFI = listingFeePaid * amount / listing.amount.)

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L179
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L87

BUSINESS IMPACT

Anyone can spam misleading zero-amount ItemSold events against any live listing, potentially corrupting off-chain sale analytics and indexers. There is no fund impact.

RECOMMENDED NEXT STEP

Validate that the purchase quantity is greater than zero at the start of the buy function.

RETEST

Fixed. buyItem now reverts on amount == 0 (Carbonfi_Marketplace.sol), blocking zero-amount event spam. (commit 5e10d0a)

I005 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

INFO

CarbonFiRetirement setRetireFee Lacks Input Validation / Sanity Bounds

✓ FIXED

DESCRIPTION

setRetireFee(uint256 newFee) updates the retirement fee with NO validation no lower bound (> 0) and no upper cap unlike the rest of the suite, where fee setters are bounded (e.g., CarbonNFT's setMintFeePerTon enforces newFee > 0 && newFee <= MAX_FEE_PER_TON). The owner is trusted, so this is not an attacker vector; it is flagged as robustness against owner MIS-configuration and for cross-contract consistency. Setting the fee to 0 makes retirement free (and pulls a 0-value CAFI transfer, crediting 0 to the pending wallets harmless but unintended); setting it to an extreme value silently prices retirement out of reach until corrected. There is also no RetireFeeUpdated event, so fee changes are not observable off-chain.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L181
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/CarbonFi_Retire.sol#L24

BUSINESS IMPACT

No external-attacker impact; the concern is owner-misconfiguration robustness and suite consistency an accidental 0 or extreme fee silently changes retirement economics, with no event emitted to flag the change.

RECOMMENDED NEXT STEP

Add sanity bounds consistent with the rest of the suite reject a zero fee and cap it at a defined maximum and emit a fee-update event whenever the retirement fee changes so the change is observable off-chain.

RETEST

Fixed. setRetireFee now rejects zero (revert InvalidFee()) and enforces an upper cap MAX_RETIRE_FEE = 10_000e18 (revert FeeTooHigh()), and emits RetireFeeUpdated(old, new) (CarbonFi_Retire.sol). Consistent with the rest of the suite's bounded fee setters.

I006 · BEST PRACTICE

INFO

AutoStaking APY Rate Not Snapshotted setAPY Retroactively Re-Prices Existing Stakes

✓ FIXED

DESCRIPTION

calculateReward always reads the current apyRates[periodIndex]; StakeInfo stores no per-stake rate. setAPY lets the owner change a lock period's APY at any time, and the new rate applies retroactively to all existing, already-locked stakes and so cannot be exited even if the staker disagrees with the new rate. The owner is a trusted party under this engagement's ground rule, so this is NOT an external-attacker vector or a standalone vulnerability; it is logged for robustness/consistency. As written, a staker's effective yield is not fixed by the terms accepted at stake time: an APY change (or an accidental mis-set) silently re-prices every open position, up or down, including rewards already accruing.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L153
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L278
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L31

BUSINESS IMPACT

No external-attacker impact (owner-trusted); the concern is robustness and fairness existing stakes are re-priced by any rate change, so stakers cannot rely on the APY they committed under.

RECOMMENDED NEXT STEP

Snapshot apyRates[periodIndex] into StakeInfo at stake() time and use the stored rate in calculateReward, OR scope setAPY to affect only stakes created after the change. Either fixes the accepted rate for the life of a stake.

RETEST

Fixed. StakeInfo now carries snapshotAPY, captured at stake() time from apyRates[periodIndex] (Staking.sol) and used in _calculateAccruedRewards. A later setAPY no longer re-prices existing positions; each stake keeps the rate accepted at stake time.

I007 · DATA INTEGRITY

INFO

✓ FIXED

Marketplace ItemSold Emits pricePerItem = 0 on Full Buyout (Event Field Read After delete)

DESCRIPTION

In buyItem, listing is a storage reference to listings[nftContract][tokenId][seller]. On a full buyout (listing.amount reaches 0) the listing slot is deleted before the ItemSold event reads listing.pricePerItem. Because listing points at the now-zeroed storage slot, listing.pricePerItem reads back as 0, so ItemSold reports a sale price of 0 for every listing that is fully bought out. Partial buys are unaffected (no delete runs, so the field is still populated). There is no fund or accounting impact the USDT settlement uses totalPriceUSDT computed earlier from the correct price; only the emitted event field is corrupted. Any off-chain indexer, price-history feed, or analytics pipeline consuming ItemSold will record full-buyout sales at price 0. here the issue is event-data integrity.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L185
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L204
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Carbonfi_Marketplace.sol#L207

BUSINESS IMPACT

Every full-buyout sale emits ItemSold with pricePerItem = 0, polluting off-chain sale history, indexers, and analytics. There is no on-chain fund, escrow, or accounting impact.

RECOMMENDED NEXT STEP

Cache the listing's price into a local variable before the listing slot is deleted, and emit that cached value in the sale event.

RETEST

Fixed. pricePerItem is cached into a local before the delete, and ItemSold emits the cached value (Carbonfi_Marketplace.sol). (commit 5e10d0a)

I008 · CODE QUALITY

INFO

AutoStaking Minor Issues (onlyEOA Anti-Pattern, Dead Code, Reward-Token Footgun)

✓ FIXED

DESCRIPTION

- onlyEOA uses require(tx.origin == msg.sender) to gate stake/claimReward/compoundReward, blocking smart contracts, multisigs, and smart-contract wallets; tx.origin-based access control is a discouraged anti-pattern (breaks account abstraction and composability).
- estimateMintGas() is dead leftover code returning a hardcoded constant.
- stakes[user] is an unbounded array; getActiveStakes and per-index access grow with the number of stakes (heavy view / gas creep for very active users).
- updateRewardToken lets the owner change rewardToken while pendingRewardWithdrawals and rewardPoolBalance were denominated/funded in the previous token, so pending rewards would be paid in the new token (owner-trusted, but a footgun).

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L64
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L350
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L320
- https://github.com/aresa1801/cafi-contract/blob/e50ab5cfda1fb59875372e72a9c718e3f443bd01/Staking.sol#L289

BUSINESS IMPACT

No direct security impact; the items represent a composability limitation (the tx.origin gate), a maintainability concern (dead code and unbounded enumeration), and an admin footgun (updateRewardToken).

RECOMMENDED NEXT STEP

Reconsider/remove the tx.origin EOA gate (rely on nonReentrant if the goal is anti-flashloan); remove estimateMintGas; bound or paginate stake enumeration; restrict or guard updateRewardToken against changing while obligations are outstanding.

RETEST

Fixed. Remaining items resolved getActiveStakes returns only active stakes; the per-user cap uses activeStakeCount (active, not historical); and updateRewardToken reverts while any liability exists (totalStaked/rewardPoolBalance/totalPendingWithdrawals must all be 0) (commit 5e10d0a).

I009 · CODE QUALITY

getActiveStakes Returns Inactive Stakes (Misnamed)

INFO

✓ FIXED

DESCRIPTION

getActiveStakes returns the user's entire stakes array, including withdrawn (inactive) positions. Integrators relying on the name may double-count or display closed positions as active.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L342-L344

BUSINESS IMPACT

Off-chain integrators may misrepresent a user's active position set. No on-chain impact.

RECOMMENDED NEXT STEP

Filter to isActive == true entries, or rename the function to reflect that it returns all historical stakes.

RETEST

Fixed. getActiveStakes now returns only isActive entries (Staking.sol). (commit 5e10d0a)

I010 · ACCESS CONTROL (SCWE-016)

renounceOwnership Not Disabled on Staking / Marketplace / DEX

INFO

✓ FIXED

DESCRIPTION

CarbonFi and CarbonNFT override renounceOwnership to revert, but AutoStaking, CarbonFiMarketplace, and CarbonDEX_USDT_Signature inherit OpenZeppelin's default, which lets the owner set ownership to address(0). (The retirement contract is also unprotected see I002.) These contracts hold owner-only emergency and configuration levers, so an accidental or malicious renounce permanently bricks pausing, fee configuration, reward funding, and emergency recovery.

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Staking.sol#L26
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Carbonfi_Marketplace.sol#L24
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/CarbonDEX.sol#L25

BUSINESS IMPACT

Irreversible loss of all admin control if ownership is renounced on these contracts.

RECOMMENDED NEXT STEP

Override renounceOwnership() to revert on all three contracts, consistent with CarbonFi/CarbonNFT.

RETEST

Fixed. renounceOwnership is overridden to revert on AutoStaking (Staking.sol), CarbonFiMarketplace (Carbonfi_Marketplace.sol), and CarbonDEX (CarbonDEX.sol). (commit 5e10d0a)

I011 · CODE QUALITY / BEST PRACTICE

INFO

Additional Minor & Informational Observations

✓ FIXED

DESCRIPTION

Two low-impact observations found during the retest; neither has a direct fund-loss or attacker vector.

- No Pausable on CarbonFiRetirement the contract inherits only Ownable2Step and ReentrancyGuard, so retireNFT and withdrawPendingFees cannot be halted during an incident, unlike the companion CarbonNFT which is Pausable. (CarbonFi_Retire.sol)
- Free retire / batchRetire bypasses the fee + certificate flow CarbonNFT.retire/batchRetire burn a holder's credits with no CAFI fee and no certificate, and these burns are not counted in CarbonFiRetirement.totalRetiredByTokenId. The official retirement path (CarbonFiRetirement.retireNFT, which charges retireFee and writes a certificate) can therefore be bypassed, and the on-chain "total retired" metric under-counts any credits destroyed via the free path. (Mint_NFT.sol vs CarbonFi_Retire.sol)

AFFECTED CODE

- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/CarbonFi_Retire.sol#L24
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/Mint_NFT.sol#L278-L295
- https://github.com/aresa1801/cafi-contract/blob/53eaa74ba3aeb3dd63328f42cdb31be1d2e621fd/CarbonFi_Retire.sol#L119-L177

BUSINESS IMPACT

No direct security or fund impact. (1) is a robustness/consistency gap with no emergency stop for retirement during an incident. (2) is a fee-revenue and accounting-integrity gap; retirements can occur without a fee, without a certificate, and without updating the protocol's retired-supply counter.

RECOMMENDED NEXT STEP

(1)Add OpenZeppelin Pausable to CarbonFiRetirement and gate retireNFT/withdrawPendingFees with whenNotPaused. (2) Route all retirements through the fee+certificate contract, or charge the fee and record a certificate (and increment the retired counter) inside CarbonNFT.retire/batchRetire, or restrict those burns to the retirement contract.

RETEST

Fixed. closed by dead-coding the bypass path. CarbonNFT.retire/batchRetire are now gated by onlyRetirementContract (Mint_NFT.sol), modifier , so an ordinary holder can no longer call them to free-burn credits with no fee and no certificate; the only retirement path left for users is CarbonFiRetirement.retireNFT, which charges retireFee and writes a certificate. The fix works by rendering the two functions effectively dead code: the sole permitted caller the registered retirementContract (CarbonFiRetirement) never invokes them (retireNFT burns via safeTransferFrom to 0x…dEaD at CarbonFi_Retire.sol, not via CarbonNFT.retire()), and no in-scope contract calls retire/batchRetire at all. They are therefore unreachable, which is acceptable: the free-burn bypass is closed and the fee/certificate flow is unaffected. (No security impact from the dead code; optionally, routing retireNFT through CarbonNFT.retire() would make them live and also let expired credits be burned.) (commit 6fe5d98)

DISCLOSURE

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or issue and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

REVIEWER(S) & CONTACT

Shreyas Koli
Security Auditor II

Prasad Kuri
Security Auditor

Neel Shah
Security Auditor

LEAD REVIEWERS & REPORT SIGNATORIES



Shashank
Co-founder & CEO · CredShields

Security analyst and penetration tester with **12 years of experience** in pentesting and **5 years** in managing bug bounty programs. Listed in **Google, Facebook, Apple,** and 30+ websites' hall of fame for reporting valid security issues.

CVE-2017-7063 CVE-2017-7062 CVE-2017-2458



Aditya Dixit
Audit & Research Lead · CredShields

Senior security researcher and penetration tester with **10+ years** in cybersecurity. Led pentest teams at **Cobalt and HackerOne**; previously at Binary.com. Executed **500+ pentests** across banking, fintech, payments, healthcare, telecom, and media, covering web, mobile (iOS/Android), network, cloud, code review, and smart contracts.

OSCP CREST CPSA AWS Security Specialty AWS Cloud Practitioner Qualys VMDR